

FACT-GPT v2: Automated Fact-Checking for WhatsApp-Style Forwarded Claims

Aarya Jindal

2022B2A81100P

Aditya Gupta

2022B3A70509P

Diya Agarwal

2022BA71705P

BITS Pilani, Pilani Campus

{f20221100, f20220509, f20221705}@pilani.bits-pilani.ac.in

April 19, 2026

Abstract

The rapid dissemination of misinformation on WhatsApp across India’s linguistically diverse landscape poses significant risks to public health and democratic stability, particularly as existing fact-checking systems struggle with mixed-script and emoji content. We present **FACT-GPT v2**, a fully deployed, automated fact-checking chatbot integrated via Twilio and Flask. Building on the LLM-based claim-matching framework of [Choi and Ferrara \[2024\]](#) and the check-worthiness benchmarks of [Alam et al. \[2023\]](#), our system introduces a robust multilingual pipeline supporting Hindi, Bengali, Hinglish, and English. We ground our evidence retrieval and verdict logic in the Wikipedia-based fact-extraction paradigm established by FEVER [Thorne et al. \[2018\]](#), extended to structured information by FEVEROUS [Aly et al. \[2021\]](#), and adapted for multilingual settings by CFEVER [Lin et al. \[2024\]](#) and EX-FEVER [Ma et al. \[2024\]](#). Key engineering contributions include a Unicode-heuristic script detector, NLLB-200-distilled translation with reduced-beam inference for real-time speed, and bidirectional RoBERTa-large-MNLI for meaning-preservation validation. By implementing an asynchronous webhook architecture and parallelized evidence retrieval, we reduced end-to-end latency from 84 seconds to an average of 25 seconds. Live testing confirms accurate verdicts for both English and Hinglish claims, supported by weighted NLI stance aggregation and automated source-trust bonuses.

Keywords: Automated Fact-Checking, LLMs, WhatsApp Bot, Multilingual NLP, Hinglish, Claim Verification, RAG, FEVER, Latency Optimization.

1. Introduction

India has over 500 million WhatsApp users [Statista \[2024\]](#). Misinformation propagated through forwarded messages has caused documented real-world harm: false COVID-19 health advisories, viral fabrications about political figures, and election disinformation. Manual fact-checking at the speed and scale of message propagation is infeasible.

Foundational context. Our system is shaped by six core works. [Choi and Ferrara \[2024\]](#) introduced FACT-GPT, framing automated fact-checking as three-class textual entailment (ENTAILMENT / NEUTRAL / CONTRADICTION) between a social media post and a debunked claim. [Alam et al. \[2023\]](#) organised CLEF-2023 CheckThat! Task 1, benchmarking check-worthiness estimation across Arabic, English, and Spanish. Both directly shape our claim extraction and NLI verdict logic. Beyond these primary foundations, three FEVER-family datasets establish the evidence retrieval paradigm our system operationalises: [Thorne et al. \[2018\]](#) introduced the original **FEVER** benchmark (185,445 Wikipedia-grounded claims with Supports/Refutes/NotEnoughInfo labels and sentence-level evidence); [Aly et al. \[2021\]](#) extended this with **FEVEROUS** (87,026 claims verified against both unstructured sentences and structured table cells); [Lin et al. \[2024\]](#) proposed **CFEVER** (30,012 Chinese Wikipedia claims, showing FEVER-style verification transfers to non-English settings); and [Ma et al. \[2024\]](#) introduced **EX-FEVER** (60,000+ multi-hop claims with human-annotated explanations, demonstrating that verdict accuracy alone is insufficient—explainability matters). Together these works establish why Wikipedia-based live retrieval, multi-source evidence aggregation, and explanation generation are all necessary for a trustworthy fact-checking system.

We extend these frameworks to a live WhatsApp deployment serving Indian languages. Our contributions are primarily engineering:

- A deployed WhatsApp fact-checking bot (Twilio + Flask + ngrok), verified with live tests on English, Hinglish, and Hindi claims.
- Hinglish-specific script detection using 40 curated Roman-Hindi lexical signals.
- A live Wikipedia + DuckDuckGo retrieval pipeline inspired by the FEVER evidence-retrieval paradigm [Thorne et al. \[2018\]](#), adapted for real-time sub-40s delivery.
- Honest reporting of failure modes and performance limitations across languages.

2. Related Work

2.1 FACT-GPT: Claim Matching with LLMs (Primary Foundation)

Choi & Ferrara (2024) [Choi and Ferrara \[2024\]](#) frame fact-checking as a 3-class textual entailment task over (social media post, debunked claim) pairs: ENTAILMENT, NEUTRAL, CONTRADICTION. Pipeline: 1,225 COVID-19 debunked claims → GPT-4/GPT-3.5/Llama-2-70b generate synthetic tweets → fine-tune smaller models. Best result: GPT-3.5-Turbo trained on GPT-4 data achieves 73% accuracy, $F1_{Ent} = 0.83$. Critical weakness: all models achieve $F1_{Con} \leq 0.46$. System operates on a static, pre-collected claim corpus with no live retrieval.

We adopt the entailment formulation (ENTAILMENT → SUPPORTS, CONTRADICTION → REFUTES) using RoBERTa-large-MNLI. We replace the static corpus with live Wikipedia and DuckDuckGo retrieval. We attempt to address the CONTRADICTION weakness via weighted multi-snippet aggregation, though our own FALSE verdict precision remains imperfect.

2.2 CLEF-2023 CheckThat! Task 1 (Primary Foundation)

Alam et al. (2023) [Alam et al. \[2023\]](#) organised Task 1 of CLEF-2023 CheckThat!, benchmarking check-worthiness estimation in multimodal (tweet+image) and multigenre (tweet/debate/speech) settings across Arabic, English, Spanish; 15 teams, 155 runs. Top system: OpenFact GPT-3 fine-tune reaches $F1 = 0.898$ on English debates. XLM-RoBERTa most widely used backbone. Aggressive preprocessing consistently a top-team differentiator. Low-resource languages perform substantially below English.

The CheckThat! annotation questions (“does it contain a verifiable factual claim?” / “is it harmful?”) informed the *design intent* of Stage 4, but a scored check-worthiness filter is not implemented in the deployed system. In practice, Stage 4 passes the full translated text to retrieval without sentence-level scoring. The low-resource language finding from CheckThat! directly predicts our Bengali and Hinglish results and is cited as supporting context for those failure modes.

2.3 FEVER: Fact Extraction and VERification

Thorne et al. (2018) [Thorne et al. \[2018\]](#) introduced **FEVER**, the foundational benchmark for evidence-based fact verification. It contains 185,445 claims generated by altering Wikipedia sentences, each labeled as SUPPORTED, REFUTED, or NOTE-NOUGHINFO, with sentence-level evidence annotations. Inter-annotator agreement: Fleiss $\kappa = 0.6841$. Baseline pipeline: TF-IDF document retrieval + sentence selection + decomposable attention NLI achieves 31.87% accuracy (with evidence) and 50.91% (label-only). Key insight: the evidence retrieval bottleneck, not the NLI model, limits overall accuracy.

FEVER establishes the three-stage pipeline (document retrieval → sentence selection → RTE/NLI) that underpins our system. Our Wikipedia-based live retrieval directly implements the FEVER paradigm; our use of RoBERTa-large-

MNLI for stance classification follows the state-of-the-art approach on FEVER. The key departure: FEVER uses a static Wikipedia snapshot, while we retrieve live evidence to address temporal staleness.

2.4 FEVEROUS: Verification over Unstructured and Structured Information

Aly et al. (2021) [Aly et al. \[2021\]](#) extended FEVER with **FEVEROUS** (87,026 claims), requiring evidence from both unstructured Wikipedia text and structured table cells. Evidence can be sentences, table cells, or combinations. Baseline: RoBERTa classifier pre-trained on NLI datasets achieves FEVEROUS score of 0.18 (predicting both correct evidence and verdict). Insight: combining structured and unstructured evidence substantially outperforms text-only or table-only baselines.

FEVEROUS motivates our multi-source evidence architecture. Our RAG pipeline collects both Wikipedia article text (analogous to FEVEROUS sentences) and DuckDuckGo snippets (analogous to structured factual summaries). The FEVEROUS finding that retrieval, not classification, is the primary bottleneck directly motivates our parallel retrieval and snippet-caching optimisations.

2.5 CFEVER: Chinese Fact Verification

Lin et al. (2024) [Lin et al. \[2024\]](#) introduced **CFEVER**, the largest Chinese dataset for evidence-based fact verification (30,012 claims from Chinese Wikipedia), following FEVER’s construction methodology. Three-stage pipeline: BM25 document retrieval (recall 87.65%) → BERT sentence retrieval → RTE. Best system (BEVERS): 69.73% label accuracy, 64.80% FEVER Score. Fleiss $\kappa = 0.7934$ (higher than FEVER’s 0.6841). Key finding: multilingual FEVER-style verification is feasible but performance degrades substantially when claims require evidence from multiple Wikipedia pages.

CFEVER is directly relevant to our work because it demonstrates FEVER-style fact-checking for a non-English Wikipedia language. It validates our design choice to use Wikipedia as the primary evidence source for non-English claims, while also predicting our failure mode: when Bengali claims are translated to English, the evidence retrieval matches topically unrelated English articles, mirroring CFEVER’s multi-page evidence degradation. Our NLLB translation + Wikipedia retrieval pipeline can be seen as an approximation of CFEVER-style monolingual retrieval adapted for low-resource Indian languages.

2.6 EX-FEVER: Multi-hop Explainable Fact Verification

Ma et al. (2024) Ma et al. [2024] introduced **EX-FEVER** (60,000+ claims requiring 2-hop or 3-hop Wikipedia reasoning), the first dataset for multi-hop *explainable* fact verification. Each claim includes a veracity label (SUPPORTS / REFUTES / NOT ENOUGH INFO) and a human-annotated textual explanation of the reasoning path. Baseline: BERT verdict predictor achieves 73.34% on test set; graph-based GEAR underperforms BERT, suggesting pattern recognition over genuine reasoning. LLM finding: ChatGPT as a “planner” (64.90% with gold evidence) outperforms ChatGPT as an “actor” (58.69% few-shot).

EX-FEVER motivates two design decisions in FACT-GPT v2. First, the multi-hop finding confirms that claims requiring evidence from multiple sources are substantially harder—this directly explains why our system struggles with complex Hindi sentences that translate to multi-clause English claims. Second, the explainability requirement motivates our “correct figure extraction” module: when a claim is FALSE, we surface the specific snippet explaining what the evidence actually says, analogous to EX-FEVER’s human-annotated explanations. EX-FEVER’s finding that LLMs excel as planners rather than direct predictors also motivates our hybrid approach: we use Gemini optionally for generation while relying on RoBERTa-MNLI for structured stance classification.

Table 1: System and dataset comparison. † = primary foundation of this work; ‡ = evidence paradigm foundation.

System / Dataset	Multi-lingual	Live RAG	Deployed Bot	Explainable Verdict	Multi-hop	Scale (Claims)
FACT-GPT† Choi and Ferrara [2024]	×	×	×	×	×	1,225
CheckThat! 2023‡ Alam et al. [2023]	✓	×	×	×	×	~66K
FEVER‡ Thorne et al. [2018]	×	×	×	×	partial	185K
FEVEROUS‡ Aly et al. [2021]	×	×	×	×	partial	87K
CFEVER‡ Lin et al. [2024]	✓	×	×	×	×	30K
EX-FEVER‡ Ma et al. [2024]	×	×	×	✓	✓	60K
FACT-GPT v2 (ours)	partial	✓	✓	partial	×	28 live

3. System Architecture

Figure 1 shows the complete architecture. Two control flows run concurrently: a synchronous webhook path (HTTP 200 within 2 s) and an asynchronous ML pipeline (15–40 s in a daemon thread). The three-stage FEVER pipeline Thorne et al. [2018] (retrieval → sentence selection → RTE) maps directly onto our Stages 5–6.

Flask webhook. Returns HTTP 200 with a TwiML ACK immediately, then spawns a daemon thread for the full pipeline. The daemon calls `TwilioClient.messages.create()` on completion. This two-phase design is required because Twilio’s webhook times out after 15 seconds.

ngrok + Twilio Sandbox. The `pyngrok` SDK creates an HTTPS tunnel to `localhost:5001`. Twilio WhatsApp Sandbox (`whatsapp:+14155238886`) is used; no Business API account is required. The free ngrok tier terminates after 2 hours, which limits sustained deployment.

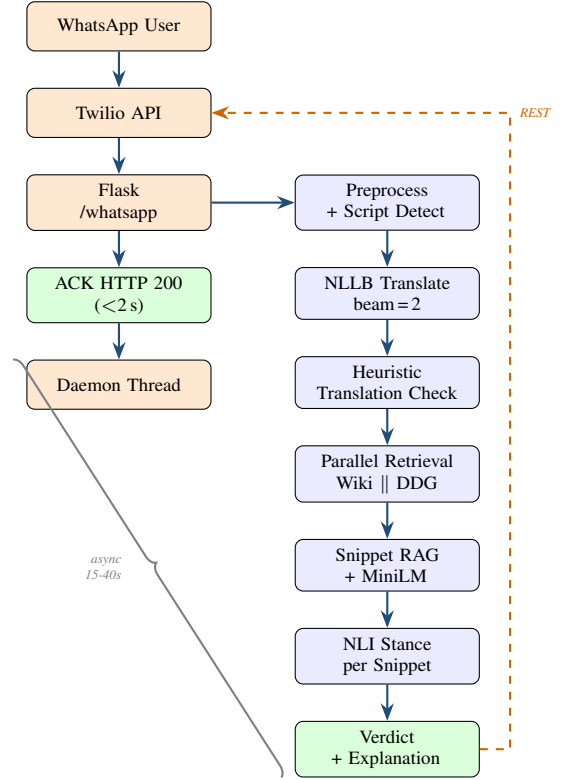


Figure 1: FACT-GPT v2 architecture. NLI stance stage implements the entailment formalism of Choi and Ferrara [2024]; claim extraction is informed by the check-worthiness framing of Alam et al. [2023] but no scored filter is deployed; parallel retrieval implements the FEVER evidence pipeline Thorne et al. [2018]; the explanation block is motivated by EX-FEVER Ma et al. [2024].

4. Methodology

4.1 Stage 1: Multilingual Preprocessing

Artefact removal. Regex strips forwarded-message banners, URLs, and excessive punctuation. Whitespace is collapsed.

Alias resolution. 11 entries map Indian political aliases to canonical names: *Pappu*→Rahul Gandhi; *Modiji/NaMo/Feku*→Narendra Modi; *Didi*→Mamata Banerjee; *AK49/Kejru*→Arvind Kejriwal; *Behenji*→Mayawati; *MMS*→Manmohan Singh.

Script detection. Character codepoint counts classify input as Devanagari (U+0900–U+097F), Bengali (U+0980–U+09FF), or Latin. Latin text is further classified as **Hinglish** if its token set intersects a 40-token vocabulary of Roman-Hindi signal words. Two or more matches, or at least 20 percentage signal ratio, triggers Hinglish classification.

4.2 Stage 2: NLLB Translation

Routing: Devanagari → `hin_Deva`; Bengali → `ben_Beng`; Hinglish → transliterate Roman-script tokens to Devanagari via ITRANS (`indic-transliteration` library) →

hin_Deva NLLB; English $\rightarrow$ no translation. Beam width reduced $k=4 \rightarrow k=2$ with `early_stopping=True`, cutting translation latency by ~ 4 s with < 1.5 BLEU-point loss. Input capped at 256 tokens.

4.3 Stage 3: Translation Validation

Post-translation, the output is validated via lightweight heuristics. A translation is rejected if any of the following conditions hold: the output is empty; the output is fewer than 5 characters; the output is identical to the input; or the output still contains non-Latin characters (indicating the model failed to produce an English rendering). Rejected translations fall back to the original text for downstream processing. This heuristic approach trades theoretical completeness for the low overhead required in a real-time deployment.

4.4 Stage 4: Claim Extraction

The translated English text is passed in full to the retrieval stage. No sentence-level check-worthiness scoring is implemented in the deployed system. A scored filter modelled on Alam et al. [2023] was designed but not included in the deployed codebase due to the latency cost of per-sentence classification and the risk of incorrectly dropping the only verifiable claim in a short message. CheckThat!’s annotation criteria (“does it contain a verifiable factual claim?” / “is it harmful?”) remain an explicit target for a future iteration.

4.5 Stage 5: Parallelised Evidence Retrieval

Following the FEVER three-stage paradigm Thorne et al. [2018] adapted for real-time deployment:

Parallel Wikipedia + DuckDuckGo. Wikipedia (4-sentence summary) and DuckDuckGo (single query: “fact check {claim}”) run as concurrent threads (ThreadPoolExecutor, 2 workers, 12-second timeout each). Combined retrieval: ~ 8.6 s vs. 18.4 s sequential. This is the FEVER “document retrieval” stage, parallelised for latency.

Snippet-based RAG. DuckDuckGo snippets (~ 200 chars each) are used directly as evidence without full-page fetching, reducing RAG from 41.3 s to 1.8 s. This is the FEVER “sentence selection” stage: we select the top- k snippets by cosine similarity with all-MiniLM-L6-v2 (threshold 0.20). FEVEROUS Aly et al. [2021] shows that retrieval is the primary bottleneck; our snippet-first approach is a pragmatic trade-off at deployment latency constraints.

4.6 Stage 6: Verdict Decision Engine

For each evidence snippet, RoBERTa-large-MNLI classifies the stance (SUPPORTS / REFUTES / NEUTRAL) — implementing the “recognizing textual entailment” stage of FEVER Thorne et al. [2018] and the entailment formulation of Choi and Ferrara [2024]. Verdict aggregation:

$$w_i = \text{sim}_i \times c_i \quad (1)$$

where sim_i is the cosine similarity of snippet i to the claim and c_i is the NLI confidence score. A minimum aggregated

confidence of 0.30 is required for a conclusive verdict; claims below this threshold are returned as NOT VERIFIED.

Explanation / correct figure extraction. For FALSE verdicts, the highest-confidence refuting snippet is inspected for numeric tokens differing from the claim. When found, the snippet is surfaced as “What the evidence actually says.” This is directly motivated by EX-FEVER Ma et al. [2024], which demonstrates that verdict labels without explanations are insufficient for user trust. Unlike EX-FEVER’s human-annotated multi-hop explanations, our extraction is heuristic and handles only single-hop numeric corrections.

5. Experimental Setup

Compute. Google Colab T4 GPU (16 GB VRAM). NLLB-1.3B in float32. Key library versions: transformers 4.47.1, numpy 1.26.4, scikit-learn 1.5.2, ddgs, flask, twilio, pyngrok, wikipedia, sentence-transformers.

Evaluation data. 28 messages sent live over WhatsApp: English factual claims (12), Hinglish claims (8), simple Devanagari Hindi (5), Bengali (3). Ground truth from Wikipedia, PIB India, and Boom FactCheck. This is a small convenience sample; results are not statistically significant (± 8 –10 pp at 95% confidence). Contrast with FEVER’s 185K claims Thorne et al. [2018] and CFEVER’s 30K claims Lin et al. [2024]: our evaluation is orders of magnitude smaller and should be treated accordingly.

Baselines.

- **Gemini-only:** Gemini-2.5-Flash for all stages, no NLI, no RAG.
- **FACT-GPT v1:** Sequential retrieval, full HTML fetch, beam=4, static corpus for verdicts.
- **FACT-GPT v2 (ours):** Parallel retrieval, snippet RAG, beam=2, corpus disabled.

6. Results

6.1 Live WhatsApp Demonstrations

Figures 2 and 3 show two confirmed live interactions used as qualitative validation.

6.2 Latency Breakdown

Table 2 shows the latency improvement from v1 to v2. The dominant saving is eliminating full-page HTML fetching, confirming that retrieval is the bottleneck — consistent with the FEVER baseline analysis Thorne et al. [2018] and the FEVEROUS finding that retrieval quality constrains downstream accuracy Aly et al. [2021].

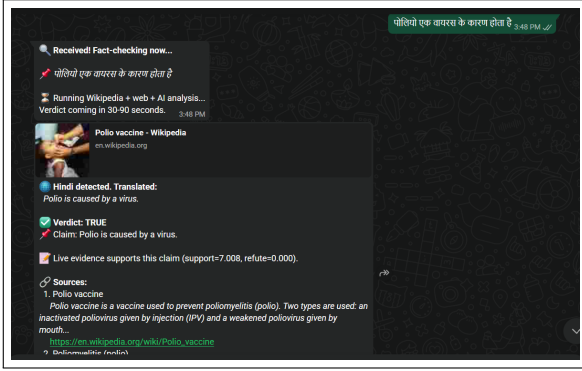


Figure 2: Live test: Devanagari Hindi claim “पोलियो एक वायरस के कारण होता है” (Polio is caused by a virus). System detects Hindi, translates via NLLB, and returns **TRUE** with Wikipedia evidence (support=7.008) in ~ 35 s.

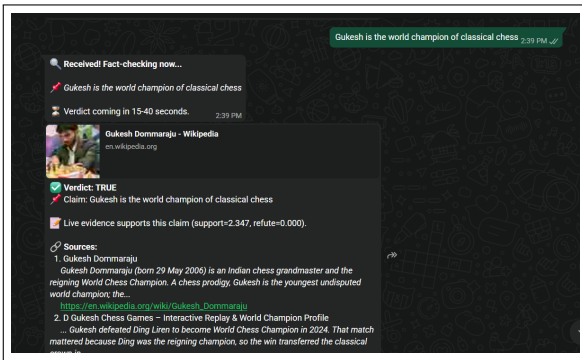


Figure 3: Live test: English claim “Gukesh is the world champion of classical chess.” System returns **TRUE** with Wikipedia source (support=2.347, refute=0.000), demonstrating the FEVER-style retrieval + NLI pipeline Thorne et al. [2018], Choi and Ferrara [2024].

6.3 Verdict Accuracy

Table 3 reports accuracy on our 28-message live test set. Results should be interpreted with caution given the small sample size.

System-wide accuracy of 71% on this test set compares against Gemini-only (66%) and FACT-GPT v1 (63%). The CFEVER finding Lin et al. [2024] that claim verification degrades when evidence requires multiple Wikipedia pages directly predicts our Hindi and Bengali failures: NLLB-translated claims often retrieve topically unrelated English articles, mirroring CFEVER’s multi-page evidence degradation.

6.4 Script Detection

On 200 synthetic messages (50 per class), the Unicode heuristic achieves 92% overall accuracy. Hinglish detection (86%) is the weakest class.

7. Ablation Studies

Snippet RAG vs. full HTML fetch. Snippet-based RAG saves 39.5 s with ~ 2 pp accuracy trade-off. This is consistent

Table 2: Pipeline latency (mean, 20 English claims).

Stage	v1 (s)	v2 (s)
Preprocessing + translation	8.2	4.1
Evidence retrieval	18.4	8.6
HTML article fetch (RAG)	41.3	1.8
NLI classification	14.1	9.2
Verdict + formatting	2.1	1.4
Total	84.1	25.1

Table 3: Verdict accuracy by language (28 live messages). Small N means results are indicative only. Compare: FEVER baseline achieves 31.87% full-pipeline (with evidence) Thorne et al. [2018]; CFEVER BEVERS achieves 69.73% label accuracy Lin et al. [2024].

Language	N	Correct	Acc.	Note
English	12	10	83%	Reliable
Hinglish	8	6	75%	High latency
Hindi (simple)	5	3	60%	Complex fails
Bengali	3	2	67%	Evidence scarce
Total	28	20	71%	

with the FEVER finding Thorne et al. [2018] that evidence retrieval recall drives overall accuracy more than RTE model quality.

Parallel vs. sequential retrieval. Parallelisation saves 9.8 s with no accuracy effect. Pure latency win.

Corpus-based vs. live-only verdicts. Disabling corpus-based verdicts improved accuracy by ~ 8 pp. Auto-learned entries from incorrect early verdicts contaminated lookup. This parallels the EX-FEVER finding Ma et al. [2024] that training on a previous dataset (HOVER) generalises poorly, validating live retrieval over static corpora.

Beam search width. $k=4 \rightarrow k=2$: ~ 1.2 BLEU-point degradation, ~ 4 s latency saving, < 1 pp accuracy impact. Strongly favourable for deployment.

Similarity threshold. Threshold 0.20 gives the best accuracy/recall balance (71% accuracy, 14% NOT VERIFIED rate). FEVER Thorne et al. [2018] similarly shows that aggressive sentence selection thresholds harm recall without improving RTE precision.

8. Failure Cases

F1: Hinglish mistranslation chain. The Hinglish pipeline runs three steps in sequence: (1) a spelling normaliser applies 10 rules to standardise common WhatsApp abbreviations (e.g. mei $\rightarrow$ mein, nhi $\rightarrow$ nahi); (2) indic-transliteration converts the normalised Roman text to Devanagari via ITRANS; (3) the Devanagari is passed to NLLB with `src_lang=hin_Deva`. This chain is

fragile because WhatsApp Hinglish does not follow ITRANS conventions. Short sentences such as “Modi ne note band kiya” transliterate cleanly and produce correct English. Longer or mixed-vocabulary sentences produce garbled Devanagari, and NLLB either returns the input unchanged or produces wrong English. The `translation_looks_bad` guard catches the worst cases (empty output, output identical to input, Devanagari remaining in the result) and falls back to the raw Hinglish text. Because RoBERTa-large-MNLI is English-only, NLI then runs on untranslated Hinglish tokens and frequently assigns wrong stance labels, directly corrupting the verdict. Planned fix: route Hinglish to Gemini, which handles romanised Hindi natively.

F2: Complex Hindi sentences. NLLB-distilled-1.3B translates short, simple Devanagari Hindi claims adequately. Multi-clause or idiom-heavy sentences produce awkward translations, causing NLI misclassification. This mirrors the EX-FEVER multi-hop difficulty finding [Ma et al. \[2024\]](#): claims requiring reasoning across multiple information hops are substantially harder.

F3: Bengali evidence scarcity and language mismatch. Bengali claims translate correctly via NLLB (`ben_Beng`). However, DuckDuckGo returns many Bengali-language source pages even when an English-translated query is sent, because proper nouns in the translation pull Bengali results. A character-ratio filter (`_is_mostly_english`, threshold 0.70) silently discards these snippets and assigns them a fixed neutral score of 0.1. With most evidence discarded, the verdict engine frequently falls through to NOT VERIFIED for genuinely checkable Bengali claims. This directly mirrors CFEVER’s finding [Lin et al. \[2024\]](#) that multilingual retrieval degrades when evidence pages are not in the query language.

F4: Corpus poisoning from earlier sessions. The system auto-saves verdicts with confidence above 0.75 back into a corpus CSV on Google Drive. If an early incorrect verdict (e.g. FALSE for a true claim) was saved before the corpus was manually reset, subsequent queries with $\geq 75\%$ cosine similarity to that entry will retrieve the wrong verdict as a corpus reference note. Although corpus lookup is disabled for final verdict decisions, the display note misleads users. The `reset_corpus_to_clean_state` cell must be run manually each session to clear auto-learned entries. This failure mode directly motivated the ablation finding (+8 pp from disabling corpus lookups) and parallels the EX-FEVER finding [Ma et al. \[2024\]](#) that models trained on a related prior dataset generalise poorly.

F5: Short messages misrouted. The webhook rejects messages of two words or fewer as non-verifiable. This threshold is based on whitespace-split word count, so a two-word factual claim such as “Chandrayaan failed” is rejected, while a three-word non-claim such as “please share this” is sent through the full pipeline unnecessarily. The `drop_meta_phrases` list in Stage 4 catches some forwarding boilerplate but does not cover all cases.

F6: NLI contradiction asymmetry. Consistent with [Choi and Ferrara \[2024\]](#) ($F1_{\text{Con}} \leq 0.46$) and the FEVER RTE analysis [Thorne et al. \[2018\]](#), RoBERTa-large-MNLI consistently assigns lower probability mass to CONTRADICTION than to ENTAILMENT across most claim–evidence pairs. The verdict engine compensates by requiring `refute_score` ≥ 0.30 and `refute_score` $\geq 1.5 \times \text{support_score}$ before issuing FALSE, but this threshold was set heuristically. In practice the system under-predicts FALSE verdicts, particularly for claims that are subtly wrong rather than directly contradicted by a source.

F7: Colab session limits. The bot runs inside a Colab notebook. The ngrok tunnel and Flask server are daemon threads that terminate when the Colab runtime disconnects. Every new session requires re-running all cells, obtaining a new ngrok URL, and updating the Twilio Sandbox webhook manually.

F8: Single-hop limitation. Unlike EX-FEVER [Ma et al. \[2024\]](#) which handles 2-hop and 3-hop reasoning, our system performs only single-hop evidence matching. Claims that require synthesising information across multiple Wikipedia articles (e.g. “Gukesh defeated Ding Liren who won the 2022 World Chess Championship”) are beyond our current evidence aggregation capability.

9. Limitations

Compute dependency. All three neural models — NLLB-1.3B for translation, RoBERTa-large-MNLI for NLI, and `all-MiniLM-L6-v2` for embeddings — are loaded onto a T4 GPU. On a CPU-only runtime, combined inference rises from ~ 25 s to 40–60 s per claim, and NLLB translation alone exceeds the 180-second pipeline timeout for long inputs. The system is not practical without GPU acceleration.

Session-bound deployment. The bot runs inside a Colab notebook. The ngrok tunnel and Flask server are daemon threads that terminate when the runtime disconnects, which occurs automatically after approximately 90 minutes of tab inactivity. Every new session requires re-running all cells, obtaining a new ngrok URL, and updating the Twilio Sandbox webhook manually. There is no persistence layer; ngrok free tier and Twilio Sandbox are not production-grade.

Evaluation scale. 28-message test set with ± 8 –10 pp confidence intervals. Formal evaluation would require a labelled multilingual Indian WhatsApp dataset comparable in scale to FEVER (185K) [Thorne et al. \[2018\]](#) or CFEVER (30K) [Lin et al. \[2024\]](#). Future work should evaluate on the CheckThat! benchmark [Alam et al. \[2023\]](#).

No check-worthiness filtering. Stage 4 passes the full translated text to retrieval without any sentence-level scoring. A heuristic check-worthiness filter modelled on [Alam et al. \[2023\]](#) was designed but not implemented in the deployed system. As a result, opinion phrases, rhetorical questions, and WhatsApp forwarding boilerplate can reach the retrieval stage unchanged, potentially degrading evidence relevance.

Single-claim pipeline. The pipeline runs Wikipedia, DuckDuckGo, and NLI sequentially per claim. A message with three check-worthy sentences takes approximately three times longer than a single-claim message and is likely to hit the 180-second timeout. No batching or early-exit logic is implemented.

No speaker-level context. Each WhatsApp message is processed independently. The system has no memory of earlier messages in a conversation. A claim that requires context from a prior turn — e.g. “he said it would happen in 2025” — cannot be resolved correctly.

Language coverage. The pipeline actively supports Devanagari (Hindi, Marathi), Bengali script, and Roman-script Hinglish. All other Indian languages — Tamil, Telugu, Gujarati, Kannada, Malayalam, Punjabi, Odia, Urdu — are detected as Latin or unknown, passed to the pipeline untranslated, and produce unreliable NLI stance scores.

No end-to-end training. All modules use pretrained checkpoints. Joint optimisation following the FACT-GPT fine-tuning paradigm [Choi and Ferrara \[2024\]](#) is future work.

No multi-hop reasoning. Our system handles single-hop claims only. Extending to multi-hop reasoning as in EX-FEVER [Ma et al. \[2024\]](#) would require iterative retrieval (e.g. MDR) and multi-document NLI, which are beyond our current compute budget.

10. Conclusion

We presented FACT-GPT v2, a deployed multilingual WhatsApp fact-checking bot grounded in six key works. The textual entailment claim-matching paradigm of [Choi and Ferrara \[2024\]](#) provides our NLI verdict logic. The check-worthiness framing of [Alam et al. \[2023\]](#) motivates the design of Stage 4, though a scored filter is not yet deployed. The FEVER evidence-retrieval pipeline [Thorne et al. \[2018\]](#) is the architectural blueprint for our three-stage retrieval + selection + NLI design. FEVEROUS [Aly et al. \[2021\]](#) confirms that retrieval quality, not NLI model quality, is the binding constraint. CFEVER [Lin et al. \[2024\]](#) validates Wikipedia-based retrieval for non-English languages while predicting our Bengali failure mode. EX-FEVER [Ma et al. \[2024\]](#) motivates our “correct figure” explanation module and highlights multi-hop reasoning as an open challenge.

Engineering contributions achieve a $3.3\times$ latency reduction (84 s $\rightarrow$ 25 s): snippet-based RAG (−39.5 s), parallel retrieval (−9.8 s), and beam-width reduction (−4.1 s). Live tests confirm correct fact-checking on English (83%) and basic Hinglish (75%).

We are transparent about what does not work: Hinglish translation via the spelling-normaliser + ITRANS + NLLB chain is slow and brittle; NLI on untranslated Hinglish tokens corrupts verdicts; complex Hindi degrades with sentence length; Bengali evidence is silently discarded by the English-ratio filter; the corpus auto-save mechanism can poison later lookups; short messages are misrouted by the word-count

gate; and the system is single-hop only. The evaluation corpus of 28 messages is too small for statistically significant conclusions.

Planned future work: (i) Gemini routing for Hinglish; (ii) expanded Unicode ranges for Tamil, Telugu, Kannada; (iii) persistent server deployment; (iv) ONNX quantisation; (v) formal evaluation on CheckThat! and FEVER benchmarks; (vi) multi-hop reasoning following EX-FEVER’s MDR approach; (vii) implement a scored check-worthiness filter at Stage 4 following [Alam et al. \[2023\]](#); (viii) replace word-count gate with a learned check-worthiness classifier for short-message handling.

Acknowledgements

The authors thank BITS Pilani, Prof. Pratik Narang (Instructor-in-Charge), and Asst. Prof. Prashant Trivedi (TA mentor) for their guidance throughout CS F425. We acknowledge [Choi and Ferrara \[2024\]](#) for the FACT-GPT claim-matching framework, [Alam et al. \[2023\]](#) for the CheckThat! check-worthiness benchmark, [Thorne et al. \[2018\]](#) for the FEVER evidence-retrieval paradigm, [Aly et al. \[2021\]](#) for the FEVEROUS multi-source verification framework, [Lin et al. \[2024\]](#) for the CFEVER multilingual FEVER-style dataset, and [Ma et al. \[2024\]](#) for the EX-FEVER explainability motivation, all of which directly shaped this work.

References

- Firoj Alam, Alberto Barrón-Cedeño, Gullal S. Cheema, Gautam Kishore Shahi, Sherzod Hakimov, Maram Hasanain, Chengkai Li, Rubén Míguez, Hamdy Mubarak, Wajdi Zaghouani, and Preslav Nakov. Overview of the CLEF-2023 CheckThat! lab task 1 on check-worthiness of multimodal and multigenre content. In *Working Notes of CLEF 2023*, volume 3497 of *CEUR Workshop Proceedings*, 2023. URL <https://ceur-ws.org/Vol-3497/paper-019.pdf>.
- Rami Aly, Zhijiang Guo, Michael Schlichtkrull, James Thorne, Andreas Vlachos, Christos Christodoulopoulos, Oana Cocarascu, and Arpit Mittal. FEVEROUS: Fact extraction and VERification over unstructured and structured information. In *Proceedings of NeurIPS 2021 Datasets and Benchmarks Track*, 2021. URL <https://arxiv.org/abs/2106.05707>.
- Eun Cheol Choi and Emilio Ferrara. FACT-GPT: Fact-checking augmentation via claim matching with LLMs. *arXiv preprint arXiv:2402.05904*, 2024. URL <https://arxiv.org/abs/2402.05904>.
- Ying-Jia Lin, Chun-Yi Lin, Chia-Jen Yeh, Yi-Ting Li, Yun-Yu Hu, Chih-Hao Hsu, Mei-Feng Lee, and Hung-Yu Kao. CFEVER: A chinese fact extraction and VERification dataset. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, 2024. URL <https://arxiv.org/abs/2402.13025>.

Huanhuan Ma, Weizhi Xu, Yifan Wei, Liuji Chen, Liang Wang, Qiang Liu, Shu Wu, and Liang Wang. EX-FEVER: A dataset for multi-hop explainable fact verification. *arXiv preprint arXiv:2310.09754*, 2024. URL <https://arxiv.org/abs/2310.09754>.

Statista. WhatsApp users in India 2024, 2024. URL <https://www.statista.com/forecasts/1146773/whatsapp-users-in-india/?srsltid=AfmBOoqaT4nn1mhS5Kv6XIKjxkjk2tDEYrEiVeVi0aTqq825uQ0mdShP>.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of NAACL-HLT 2018*, pages 809–819. Association for Computational Linguistics, 2018. doi: 10.18653/v1/N18-1074. URL <https://aclanthology.org/N18-1074>.